

Create a Local Database from Scratch

Overview

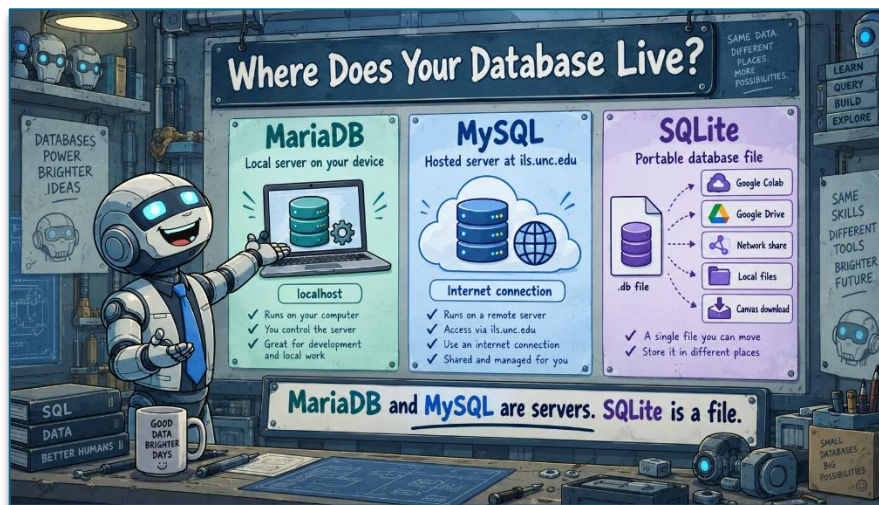
5 Sept 2026

- install and create a database
- create tables in a database
- load data into a database

Platform Update

For this module, remember:

- treat references to PostgreSQL or Postgres as references to MariaDB, unless SQLite is explicitly specified
- treat references to pgAdmin as references to DBeaver
- use MariaDB as the default database platform
- use SQLite only when an activity specifically instructs you to do so
- do not install PostgreSQL or pgAdmin for this module



ChatGPT 5.6 Sol

MariaDB Server Setup Links**local guide:**

[notes\Fall_2026\DATA715\resources\DATA715_CoursePreparation.pdf](#)

Installing MariaDB Server Guide

<https://mariadb.com/docs/server/mariadb-quickstart-guides/installing-mariadb-server-guide>

Connecting to MariaDB Guide

<https://mariadb.com/docs/server/mariadb-quickstart-guides/mariadb-connecting-guide>

CREATE TABLE Guide

<https://mariadb.com/docs/server/server-usage/tables/create-table>

Data Types Guide

<https://mariadb.com/docs/server/reference/data-types>

String Data Types Guide

<https://mariadb.com/docs/server/reference/data-types/string-data-types>

Async	
Importing Data [MariaDB]	
Importing Data	Importing data means moving information from an external source into one or more database tables. • there is no single correct way to import data • the best method depends on: ○ file format ○ amount of data ○ database location ○ available software ○ whether the import must be repeated
Data Sources	Data may come from many sources. • CSV or tab-delimited text files • Excel workbooks • SQL scripts and database backups • another database • web services and APIs • Python or other application code • manually written <i>INSERT</i> statements
Common Import Methods	DBeaver Data Import. • DBeaver provides a visual import wizard for ○ CSV ○ Excel ○ other supported file formats

	• the wizard can be used to: ○ select a source file ○ map source fields to table columns ○ configure file settings ○ review the import before loading the data DBeaver Data Import Guide: • https://dbeaver.com/docs/dbeaver/Data-import/ • the guide covers: ○ selecting a destination table ○ choosing the source file and format ○ configuring: ▪ delimiters ▪ headers ▪ quotes ▪ encoding ▪ <i>NULL</i> values ○ mapping source fields to destination columns ○ reviewing and executing the import DBeaver is often the easiest method for learning, exploring unfamiliar data, or performing a one-time import.
MariaDB Import Tools And SQL Commands	MariaDB provides commands and utilities for importing data directly into the database. • these methods are useful for: ○ larger files ○ repeatable processes ○ automation

- o situations where a graphical interface is unavailable

MariaDB Importing Data Guide:

- <https://mariadb.com/docs/server/mariadb-quickstart-guides/mariadb-importing-data-guide>
- common methods include:
 - o *LOAD DATA INFILE*
 - o *LOAD DATA LOCAL INFILE*
 - o mariadb-import command-line utility
 - o SQL scripts containing *CREATE TABLE* and *INSERT* statements
 - o database backup and restore tools
 - o custom programs written in Python or another language

Before Importing Data.

- before starting an import:
 - o inspect both the source data and destination table
 - o check:
 - column names and order
 - data types
 - required and optional columns
 - primary and foreign keys
 - date and time formats
 - character encoding
 - field delimiters
 - quoted values
 - empty values and NULL values
 - duplicate records

Import Validation	A successful import does not always mean the data was imported correctly. • after the import, validate the results by: - o checking row counts - o reviewing sample records - o looking for unexpected <i>NULL</i> values - o confirming that values were placed in the correct columns
Choosing An Import Method	Use DBeaver when you want a guided visual process or are completing a small, one-time import. • use MariaDB commands or utilities when: - o working with larger files - o automating a repeatable process - o learning how database imports operate outside a graphical tool Importing data involves more than loading a file. • it also requires: - o understanding the source - o preparing the destination - o selecting appropriate settings - o handling errors - o validating the final results
Creating Tables to Hold Data	
Creating A Table	Creating a table manually is useful for understanding how table definitions work before using more automated methods. • <i>CREATE TABLE</i> syntax is very similar across SQL databases

	- o the same general structure appears in: ▪ PostgreSQL ▪ Oracle ▪ MySQL ▪ other database systems
Planning The Table	Before creating a table, inspect the data that will be stored in it. • the table design should reflect: - o number of columns - o type of data in each column - o appropriate column names - o required constraints - o which columns make each row unique The example uses EPA vehicle data stored in a CSV file. • the file contains: - o column headers - o comma-separated values - o character data - o integer data - o floating-point numeric data
Column Naming	Source file column names may not always be valid or desirable database column names. • problems may include: - o white space - o inconsistent naming conventions - o camel case

	o names that do not follow the chosen database style Consistency is the most important naming principle.
CREATE TABLE Structure	A table definition begins with: <pre><i>CREATE TABLE table_name (</i> <i>... column definitions ...)</i></pre> - o the column definitions are placed inside parentheses • each column definition normally contains: - o column name - o data type - o optional constraints
Character Data	Character varying fields are useful when text values have different lengths. • example: <i>manufacturer VARCHAR(50) NOT NULL</i> - o <i>VARCHAR(50)</i> ▪ allows values up to 50 characters - o <i>NOT NULL</i> ▪ means every row must contain a value for that column

Integer Data	Integer data is appropriate for whole-number values. • examples include: - o model year - o production count ▪ should also be an integer because fractional vehicles are not meaningful Years can also be stored as integers.
NOT NULL	<i>NOT NULL</i> prevents a column from containing a <i>NULL</i> value. • use <i>NOT NULL</i> when a value must exist for every row • example required fields include: - o manufacturer - o model year - o regulatory class - o vehicle type Do not apply <i>NOT NULL</i> when missing values should be allowed.
Numeric Data	Numeric data is used when values contain decimal places. • the <i>NUMERIC</i> type can define both precision and scale - o precision: ▪ maximum total number of digits - o scale: ▪ maximum number of digits after the decimal point - o Example: <i>NUMERIC(8,5)</i>

- allows up to 8 total digits
- allows up to 5 digits after the decimal point

Different Precision And Scale.

- different numeric columns may require different precision and scale values
- examples from the vehicle data include:
 - o miles per gallon
 - o vehicle weight
 - o horsepower

- example definitions:

NUMERIC(7,3)

- o maximum of 7 total digits
- o 3 digits after the decimal point

NUMERIC(8,3)

- o maximum of 8 total digits
- o 3 digits after the decimal point

Allowing *NULL* Values.

- some numeric values may not always be available
 - o for example:
 - a vehicle may not yet have a calculated city miles-per-gallon value
 - leaving out *NOT NULL* allows the row to be stored even when that value is missing

Separating Column Definitions	Each column definition is separated by a comma. • forgetting a comma is a common source of <i>CREATE TABLE</i> syntax errors
Primary Keys	A primary key uniquely identifies each row in a table. • a primary key can be defined when the table is created ◦ it can also be added later The example uses a compound primary key. • a compound primary key uses more than one column to uniquely identify a row • the example primary key contains: ◦ manufacturer ◦ model year ◦ regulatory class ◦ vehicle type Example structure: <pre>PRIMARY KEY (manufacturer, model_year, regulatory_class, vehicle_type)</pre>
Finishing The Statement	The <i>CREATE TABLE</i> statement ends with: <pre>)</pre> ▪ followed by a semicolon

Creating The Table	After the statement is executed successfully, the table may not immediately appear in the database interface. • refresh the tables list to display the newly created table
Table Naming	PostgreSQL has an important behavior involving uppercase letters in table names. • using uppercase letters: ◦ can require the table name to be enclosed in double quotes in later queries • this creates unnecessary complexity • recommended PostgreSQL practice: ◦ use lowercase table names ◦ avoid uppercase characters ◦ use underscores to separate words
Data Types	Choosing the correct data type may be difficult when first creating tables. • PostgreSQL supports many different data types • common types include: ◦ <i>INTEGER</i> ◦ <i>NUMERIC</i> ◦ <i>VARCHAR</i> ◦ <i>CHARACTER</i> • PostgreSQL also supports more specialized types such as: ◦ JSON ◦ currency-specific data types

	- the PostgreSQL documentation: - provides the complete set of supported data types. With experience, choosing appropriate data types becomes easier.
Importing Data into Created Tables	
Before Importing Data	Before importing data, inspect the source file carefully. - databases expect incoming values to match the structure and data types defined in the destination table - unexpected values can cause an import to fail - check the source data for: - column names - column order - data types - missing values - unexpected text - date formats - duplicate records - values that do not match the destination table
Importing With DBeaver	DBeaver provides a guided interface for importing data into MariaDB tables. - the general process is: - select the destination table - right-click the table - choose Import Data

	- o select the source format - o choose the source file - o configure the import settings - o map source columns to destination columns - o review the settings - o run the import For CSV imports, DBeaver supports settings for: - o delimiters - o headers - o quotes - o escape characters - o <i>NULL</i> values - o encoding - o date formats
Selecting The Source File	Choose the file containing the data to be imported. • common formats include: - o CSV - o tab-delimited text - o other formats supported by DBeaver The source file should be compatible with the structure of the destination table.
Header Row	A header row contains the names of the columns. • if the source file has a header: - o configure DBeaver to use the first row as column names

	• if the source file does not have a header: - o configure the import so the first row is treated as data Incorrect header settings can cause a data row to be skipped or column names to be imported as data.
Delimiter	The delimiter identifies where one field ends and the next begins. • common delimiters include: - o comma - o tab - o pipe • CSV files normally use a comma The configured delimiter must match the source file.
Quoted Values	Text values may be surrounded by quotation marks. • quotes allow a value containing characters such as commas to be treated as a single field - o example: ▪ "Charlotte, NC" - o without quotes, the comma could incorrectly be interpreted as a field delimiter

Escape Character	An escape character allows special characters to appear inside a value without being interpreted as part of the file structure. this is useful when text contains quotation marks or other characters that would otherwise have special meaning
NULL Values	The import process must distinguish between: <i>NULL</i> - an empty string - a literal text value DBeaver allows a specific value to be configured as the NULL marker and can also convert empty strings to NULL when needed.
Column Mapping	Map each source column to the appropriate MariaDB table column. - check that: - column names correspond correctly - column order is correct - source values are compatible with destination data types - required destination columns receive values A successful mapping does not guarantee that every value is valid. - the actual data must still conform to the table definition
Import Failure	An import may fail when a source value cannot be converted to the destination column's data type. - for example: <i>model_year</i>

	- o is defined as <i>INTEGER</i> • but the source contains: - o <i>prelim 2022</i> MariaDB cannot store the entire string <i>prelim 2022</i> as an integer. • the source data or table design must therefore be corrected before the import can succeed
Investigating The Error	When an import fails, identify: • which column caused the error • which value caused the error • what data type the destination column expects • whether the source contains a recurring pattern In the EPA vehicle example, the source contains preliminary 2022 values. • the text <i>prelim</i> communicates useful information • simply deleting it would make the value compatible with <i>INTEGER</i> but would also discard information
Preserving Information During Cleaning	Instead of deleting meaningful information, separate it into another field. • the original value: - o <i>prelim 2022</i> • can be represented as: - o <i>model_year</i> = 2022 - o <i>prelim</i> = <i>y</i>

	This preserves the preliminary status while keeping <i>model_year</i> compatible with an integer data type.
Adding A Preliminary Column	Add a new column to the MariaDB table before importing the cleaned file. - Example:<pre>ALTER TABLE epa_cars ADD COLUMN prelim CHAR(1);</pre> MariaDB supports adding columns to an existing table with ALTER TABLE and ADD COLUMN. - the field only needs to store: <i>y</i> or <i>n</i> - so <i>CHAR(1)</i> is sufficient
Cleaning The Source Data	Create a prelim field in the source file. - for each row: - set prelim to <i>y</i> when the model year is preliminary - set prelim to <i>n</i> otherwise - then replace: <i>prelim 2022</i> - with: 2022 The resulting data keeps both pieces of information in fields with appropriate data types.

Saving The Cleaned Data	Save the cleaned dataset as a new file. • this preserves the original source data • the cleaned file can then be used for the database import
Retrying The DBEaver Import	Run the import again using the cleaned source file. • check the column mapping ◦ the file now contains the additional <i>prelim</i> column • map: ◦ <i>model_year</i> to <i>model_year</i> ◦ <i>prelim</i> to <i>prelim</i> • then review the remaining mappings and execute the import
Validating The Import	A completed import should always be validated. • example: ◦ <i>SELECT *</i> ◦ <i>FROM epa_cars;</i> • check that: ◦ rows were imported ◦ <i>model_year</i> contains valid integer values ◦ <i>prelim</i> contains the expected <i>y</i> or <i>n</i> values ◦ columns contain the correct source data ◦ unexpected <i>NULL</i> values were not introduced
Checking Row Counts	Compare the number of imported rows with the expected number of source records. • example: ◦ <i>SELECT COUNT(*)</i> ◦ <i>FROM epa_cars;</i>

	• a matching row count is useful evidence that the import completed correctly • it does not by itself prove that every value was imported correctly
Importing With MariaDB SQL	MariaDB also provides <i>LOAD DATA INFILE</i> for importing data directly from text files. • this is useful when: - o working with larger files - o creating repeatable import processes - o automating data loading - o working without a graphical interface MariaDB documents <i>LOAD DATA INFILE</i> as its high-speed SQL mechanism for loading rows from text files into a table. • Basic <i>LOAD DATA INFILE</i> • A basic import has the form: - o <i>LOAD DATA INFILE '/path/epa_cars.csv'</i> - o <i>INTO TABLE epa_cars;</i> The file must be accessible in the environment expected by the MariaDB server. • <i>LOAD DATA LOCAL INFILE</i> - o reads the file from the client side rather than requiring the MariaDB server to access the file directly - o example: ▪ <i>LOAD DATA LOCAL INFILE '/path/epa_cars.csv'</i> ▪ <i>INTO TABLE epa_cars;</i>

	• whether <i>LOCAL</i> imports are available depends on the MariaDB and client configuration
Importing A CSV File With SQL	A CSV import can define the delimiter, quote character, and header behavior. • Example: ◦ <i>LOAD DATA LOCAL INFILE '/path/epa_cars.csv'</i> ◦ <i>INTO TABLE epa_cars</i> ◦ <i>FIELDS TERMINATED BY ','</i> ◦ <i>ENCLOSED BY '"'</i> ◦ <i>LINES TERMINATED BY '\n'</i> ◦ <i>IGNORE 1 LINES;</i> • <i>FIELDS TERMINATED BY</i> ◦ identifies the column delimiter • <i>ENCLOSED BY</i> ◦ identifies the character surrounding quoted values • <i>LINES TERMINATED BY</i> ◦ identifies how records are separated • <i>IGNORE 1 LINES</i> ◦ skips the header row MariaDB <i>LOAD DATA</i> supports control over field formatting, line formatting, and column mapping during bulk imports.

Choosing An Import Method	Use DBeaver when you want: • a guided visual process • interactive column mapping • easy inspection of import settings • a small or one-time import use <i>LOAD DATA INFILE</i> or <i>LOAD DATA LOCAL INFILE</i> when you want: • repeatable SQL • automation • larger bulk imports • a process that does not depend on a graphical interface
Key Import Principle	Importing data is not simply moving a file into a database. • it requires: - o inspecting the source - o understanding the destination schema - o matching data types - o preserving meaningful information during cleaning - o mapping fields correctly - o handling import errors - o validating the final result A failed import often identifies a mismatch between the structure of the source data and the structure expected by the database.

Testing That Data Are Correctly Loaded	
Why Validate Imported Data	A successful import does not guarantee that the data was loaded correctly. • a process: ◦ can complete without errors ◦ while still producing unexpected results • never assume that an import worked simply because MariaDB or DBeaver reported success
Check The Expected Row Count	The first validation step is to determine how many rows should exist in the destination table. • the EPA source file contains 5,281 total rows ◦ one row is the header ◦ the expected number of data rows is therefore: ▪ 5,280 Count Rows In MariaDB. • Use COUNT(*) to determine how many rows were imported ◦ <i>SELECT COUNT(*)</i> ◦ <i>FROM epa_cars;</i> ◦ expected result: ▪ 5,280 • matching the expected row count: ◦ confirms that the expected number of records was loaded ◦ it does not prove that every value was loaded correctly

	DBeaver Row Count. • DBeaver also displays the number of rows returned by a query • running: - o <i>SELECT *</i> - o <i>FROM epa_cars;</i> - o can provide a quick visual confirmation of the number of retrieved rows - o <i>COUNT(*)</i> is still preferable when the row count itself is the validation target
Visually Inspect The Imported Data	Visual inspection is an important part of quality assurance. • review sample rows and examine each column for values that appear inconsistent with the intended data type - o look for problems such as: ▪ decimal values being truncated ▪ unexpected <i>NULL</i> values ▪ text stored in the wrong column ▪ incorrectly parsed dates ▪ values shifted between columns ▪ unexpected categories ▪ missing records
Check Numeric Precision	Numeric columns should retain the precision expected from the source data. • for example: - o a value containing decimal places:

	▪ should not lose those decimal places because the destination column was accidentally defined as <i>INTEGER</i> - o compare representative source values with their imported MariaDB values
Compare Source And Destination Queries	When transferring data from one database to another, run equivalent queries against both databases. • compare the results - o useful comparisons include: ▪ row counts ▪ minimum and maximum values ▪ category counts ▪ aggregates ▪ specific record lookups - o different results may indicate that the data was: ▪ transformed ▪ omitted ▪ duplicated ▪ incorrectly loaded

Referential Integrity	Referential integrity describes whether relationships between tables remain valid. • if one table references a record in another table: - o the referenced record should exist when the database design requires it - o for example: ▪ <i>CONDITION_OCCURRENCE</i>: • references <i>PERSON</i> through <i>person_id</i> - o a diagnosis should correspond to an existing patient - o a patient does not necessarily need to have a diagnosis
Checking For Diagnoses Without Patients	A LEFT JOIN can be used to identify condition records that do not have a matching patient. • <i>SELECT COUNT(*)</i> • <i>FROM condition_occurrence c</i> • <i>LEFT JOIN person p</i> • <i>ON c.person_id = p.person_id</i> • <i>WHERE p.person_id IS NULL;</i> - o expected result: ▪ 0 - o a result of zero: ▪ means that every <i>person_id</i> in <i>CONDITION_OCCURRENCE</i> also exists in <i>PERSON</i>

Why Use A LEFT JOIN	The goal is to preserve every row from <code>CONDITION_OCCURRENCE</code>. • the <i>LEFT JOIN</i>: - o keeps all condition records even when there is no matching <i>PERSON</i> record • <i>WHERE p.person_id IS NULL</i>: - o isolates the unmatched records This is equivalent to the PostgreSQL example that used a RIGHT JOIN with the table order reversed. • using <i>LEFT JOIN</i> often makes the validation logic easier to read
Checking The Opposite Relationship	The relationship can also be checked in the opposite direction. • <i>SELECT COUNT(*)</i> • <i>FROM person p</i> • <i>LEFT JOIN condition_occurrence c</i> • <i>ON p.person_id = c.person_id</i> • <i>WHERE c.person_id IS NULL</i>; - o this identifies: ▪ patients who do not have a matching diagnosis ▪ in this example, that is acceptable. - o patients may exist without any diagnosis records - o whether unmatched records are acceptable depends on the business rules for the database

Business Rules	Database validation should also reflect the rules of the specific dataset. • technical integrity alone does not guarantee that the data makes sense • examples of healthcare business-rule checks include: - o patient counts should not unexpectedly decrease - o patient ages should remain within plausible ranges - o expected demographic categories should not suddenly disappear - o record counts should remain within reasonable ranges - o required relationships should remain valid
Descriptive Validation	Simple descriptive queries can reveal errors that row counts do not detect. • use frequency distributions to examine whether categorical data contains the expected mix of values - o examples include: ▪ race ▪ ethnicity ▪ sex ▪ visit type ▪ diagnosis category
Why Use A LEFT JOIN	A LEFT JOIN preserves all records from PERSON. • if a <i>race_concept_id</i> does not have a corresponding record in CONCEPT, the patient still appears in the result - o the concept name will appear as <i>NULL</i>

	o this makes broken or missing concept mappings easier to detect
NULL Values	A <i>NULL</i> category is not automatically an error. • whether <i>NULL</i> values are acceptable depends on the business rules and meaning of the data - o for example: ▪ missing race data may be acceptable in a particular dataset ▪ a sudden increase in missing values may still indicate a data-loading problem
Validation Is More Than Error Checking	Data validation should combine several types of checks. • row-count validation • visual inspection • data-type validation • referential-integrity checks • business-rule checks • descriptive queries • source-to-destination comparisons Key Validation Principle. • never assume that imported data is correct because the import completed successfully • validation should confirm both structural correctness and logical correctness - o check that: ▪ the expected number of rows exists

- values appear in the correct columns
- data types preserved the intended values
- table relationships remain valid
- expected categories are present
- business rules are satisfied
- query results are consistent with the source data